

Experiment 8

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Code:

```
#include <stdio.h> #include <stdlib.h>

int main() { int i; int *arr;

// 1. malloc() - allocate memory
arr = (int *)malloc(3 * sizeof(int));

if (arr == NULL) {
    printf("Memory allocation failed!\n");
    return 1;
}

printf("Memory allocated using malloc()\n");

for (i = 0; i < 3; i++) {
    arr[i] = (i + 1) * 10;
}

printf("Values: ");
for (i = 0; i < 3; i++) {
    printf("%d ", arr[i]);
}
printf("\n");

// 2. calloc() - allocate and initialize memory
int *zeroArr = (int *)calloc(3, sizeof(int));

if (zeroArr == NULL) {
    printf("Calloc allocation failed!\n");
    free(arr);
    return 1;
}
```

```
printf("Memory allocated using calloc()\n");

printf("Initial calloc values: ");
for (i = 0; i < 3; i++) {
    printf("%d ", zeroArr[i]);
}
printf("\n");

// 3. realloc() - resize allocated memory
arr = (int *)realloc(arr, 5 * sizeof(int));

if (arr == NULL) {
    printf("Reallocation failed!\n");
    free(zeroArr);
    return 1;
}

printf("Memory resized using realloc()\n");

arr[3] = 40;
arr[4] = 50;

printf("Values after realloc: ");
for (i = 0; i < 5; i++) {
    printf("%d ", arr[i]);
}
printf("\n");

// 4. free() - release memory
free(arr);
free(zeroArr);

printf("Memory released using free()\n");

return 0;

}
```

```

ashwithreddy_earla@AshwithReddy:~$ nano Experiment8.1.c
ashwithreddy_earla@AshwithReddy:~$ gcc Experiment8.1.c -o Experiment8.1
ashwithreddy_earla@AshwithReddy:~$ ./Experiment8.1
Memory allocated using malloc()
Values: 10 20 30
Memory allocated using calloc()
Initial calloc values: 0 0 0
Memory resized using realloc()
Values after realloc: 10 20 30 40 50
Memory released using free()

```

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
#include <stdio.h> #include <stdlib.h> #include <unistd.h> #include <sys/wait.h>
```

```
#define SIZE (10 * 1024 * 1024) // 10 MB
```

```
int main() { char *data;
```

```
// Allocate 10 MB
```

```
data = (char *)malloc(SIZE);
```

```
if (data == NULL) {
    perror("malloc");
    return 1;
}
```

```
// Initialize memory
```

```
for (int i = 0; i < SIZE; i++) {
    data[i] = 'A';
}
```

```
printf("Parent: Memory allocated and initialized: 10 MB\n");
```

```
printf("Parent: PID = %d\n", getpid());
```

```
pid_t pid = fork();
```

```
if (pid < 0) {
    perror("fork");
    free(data);
}
```

```

        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("\nChild: PID = %d\n", getpid());
        printf("Child: Before modification, data[0] = %c\n", data[0]);

        sleep(2);

        // Modify memory in child
        for (int i = 0; i < SIZE; i++) {
            data[i] = 'B';
        }

        printf("Child: After modification, data[0] = %c\n", data[0]);
        printf("Child: Modified 10 MB of memory.\n");

        sleep(5);

        free(data);
        exit(0);
    }
    else {
        // Parent process
        printf("Parent: Child created with PID = %d\n", pid);
        printf("Parent: Before child modification, data[0] = %c\n",
data[0]);

        sleep(4);

        printf("Parent: After child modification, data[0] = %c\n",
data[0]);
        printf("Parent: Original data is unchanged.\n");

        wait(NULL);

        free(data);
        printf("Parent: Memory freed.\n");
    }
}

```

```
}
```

```
return 0;
```

```
}
```

```
ashwithreddy_earla@AshwithReddy:~$ gcc Experiment8.2.c -o Experiment8.2
ashwithreddy_earla@AshwithReddy:~$ ./Experiment8.2
Parent: Memory allocated and initialized: 10 MB
Parent: PID = 896
Parent: Child created with PID = 897
Parent: Before child modification, data[0] = A

Child: PID = 897
Child: Before modification, data[0] = A
Child: After modification, data[0] = B
Child: Modified 10 MB of memory.
Parent: After child modification, data[0] = A
Parent: Original data is unchanged.
Parent: Memory freed.
ashwithreddy_earla@AshwithReddy:~$ |
```